

Model Tuning

Squeeze Every Drop of Performance

■■ HYPERPARAMETER OPTIMISATION

■ LEARNING OUTCOME

By the end of this module you will master hyperparameter tuning strategies — grid search, random search, Bayesian optimisation — and build production-ready sklearn Pipelines that prevent data leakage.

■ Skills You Will Gain

- Design and run GridSearchCV and RandomizedSearchCV
- Apply Bayesian optimisation with Optuna
- Build full sklearn Pipelines with preprocessing + model
- Use StackingClassifier to combine multiple models
- Apply the right cross-validation strategy for any data type
- Prevent data leakage in end-to-end ML pipelines

■ Model tuning is where 80% accuracy becomes 87% accuracy. It's the difference between a working prototype and a production-quality model worth deploying.

SECTION 1

Cross-Validation Strategies

	When to Use	Code
	Regression or balanced classification	KFold(n_splits=5, shuffle=True)
	Classification — preserves class proportions	StratifiedKFold(n_splits=5)
	Time-ordered data — no future data leakage	TimeSeriesSplit(n_splits=5)
KFold	Reduce estimate variance — more stable	RepeatedStratifiedKFold(n_splits=5, n_repeats=3)
	Very small datasets (< 100 samples)	LeaveOneOut()
	Groups must not be split across folds	GroupKFold(n_splits=5)

SECTION 2

Grid Search vs Random Search

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV from scipy.stats import
randint, uniform # Grid Search — exhaustive (use for small param spaces) param_grid = {
'n_estimators': [100, 200, 300], 'max_depth': [3, 5, 7, None], 'min_samples_leaf': [1, 2, 5] } grid
= GridSearchCV(RandomForestClassifier(), param_grid, cv=5, scoring='f1', n_jobs=-1, verbose=1)
grid.fit(X_train, y_train) print(f'Best: {grid.best_params_} | F1: {grid.best_score_:.4f}') #
Random Search — sample from distributions (large spaces) param_dist = { 'n_estimators':
randint(100, 1000), 'max_depth': randint(3, 15), 'min_samples_leaf': randint(1, 10),
'max_features': uniform(0.3, 0.5) } rand = RandomizedSearchCV(RandomForestClassifier(), param_dist,
n_iter=50, cv=5, scoring='f1', n_jobs=-1, random_state=42) rand.fit(X_train, y_train)
```

SECTION 3

Optuna — Bayesian Optimisation

Why Bayesian Optimisation?

Grid Search: tries every combination — $O(n^k)$ trials — exponential cost Random Search: random sampling — better but ignores prior results Bayesian (Optuna): builds a probabilistic model of performance vs params — focuses next trial on PROMISING regions Result: finds better hyperparameters with 5-10x fewer trials than grid search

```
import optuna optuna.logging.set_verbosity(optuna.logging.WARNING) def objective(trial): params = {
'n_estimators': trial.suggest_int('n_estimators', 100, 1000), 'max_depth':
trial.suggest_int('max_depth', 3, 12), 'learning_rate': trial.suggest_float('learning_rate', 0.01,
0.3, log=True), 'subsample': trial.suggest_float('subsample', 0.5, 1.0),
'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0), } model =
XGBClassifier(**params, random_state=42) return cross_val_score(model, X_train, y_train, cv=5,
scoring='f1').mean() study = optuna.create_study(direction='maximize') study.optimize(objective,
n_trials=100, n_jobs=-1) print(f'Best F1: {study.best_value:.4f}') print(f'Best params:
{study.best_params}')
```

SECTION 4

sklearn Pipeline — No Data Leakage

```

from sklearn.pipeline import Pipeline from sklearn.compose import ColumnTransformer from
sklearn.preprocessing import StandardScaler, OneHotEncoder from sklearn.impute import SimpleImputer
# Preprocessing pipelines per column type num_pipe = Pipeline([ ('imputer',
SimpleImputer(strategy='median')), ('scaler', StandardScaler()) ]) cat_pipe = Pipeline([
('imputer', SimpleImputer(strategy='most_frequent')), ('ohe',
OneHotEncoder(handle_unknown='ignore', sparse_output=False)) ]) preprocessor = ColumnTransformer([
('num', num_pipe, num_cols), ('cat', cat_pipe, cat_cols) ]) # Full pipeline: preprocessing + model
pipeline = Pipeline([ ('prep', preprocessor), ('model', XGBClassifier(**best_params)) ]) #
fit_transform on train ONLY – transform applies same params to test pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test) # identical preprocessing applied

```

■ PRACTICE Q & A

Q1. Why use StratifiedKFold instead of regular KFold for classification?

A: Regular KFold may create folds with very different class distributions. StratifiedKFold preserves the original class ratio in each fold, giving unbiased performance estimates.

Q2. What is Bayesian optimisation and why is Optuna better than GridSearchCV?

A: Bayesian optimisation builds a probabilistic surrogate model of objective vs hyperparameters. It focuses future trials on promising regions rather than searching exhaustively or randomly.

Q3. What is data leakage and give a concrete example?

A: Test data information leaks into training. Example: fitting StandardScaler on all data before split — the scaler encodes test set statistics (mean, std) into the training process.

Q4. How does sklearn Pipeline prevent data leakage?

A: fit_transform() is called only on training data. When pipeline.predict(X_test) is called, it applies transform() (not fit_transform), using only statistics learned from training data.

Q5. What is model stacking?

A: An ensemble method where predictions of base models (level 0) become features for a meta-learner (level 1). The meta-learner learns which base model to trust for different inputs.

■■ Model Tuning Cheat Sheet	
<code>GridSearchCV(m, params, cv=5)</code>	Exhaustive search
<code>RandomizedSearchCV(m, dist, n_iter=50)</code>	Random sampling
<code>optuna.create_study(direction='maximize')</code>	Bayesian optimisation
<code>study.optimize(obj, n_trials=100)</code>	Run Bayesian search
<code>StratifiedKFold(n_splits=5)</code>	Balanced CV splits
<code>TimeSeriesSplit(n_splits=5)</code>	Time-ordered splits
<code>Pipeline([('prep', pp), ('model', m)])</code>	Full ML pipeline
<code>ColumnTransformer(...)</code>	Different transforms per col
<code>StackingClassifier(estimators, final)</code>	Model stacking ensemble
<code>cross_val_score(m, X, y, cv=5)</code>	Cross-validated score
<code>grid.best_params_</code>	Best hyperparameters found
<code>study.best_value</code>	Best Optuna objective value